AI Content Engine: Case Study

Nov 1, 2026 · @Ansh

TL;DR

- **Problem:** one strong long-form piece gets published once; the five channel versions never get written.
- **Built:** an n8n engine that turns one source into five channel drafts in 67 to 88 seconds, checks each one with a code-based quality gate, and queues it for human approval.
- **Result (real):** 15 of 20 test drafts passed the gate first time, none used a number missing from the source, and a short source costs about $0.12. The LinkedIn post for this project needed 20% of its words changed.

Skills shown

AI automation · Workflow automation · Prompt engineering · LLMs · Digital marketing · Complete marketing

Problem and user

One strong long-form piece (a webinar, podcast, founder interview or deep blog post) usually gets published once and dies. Each channel version is another rewrite for a different format and reader, so the LinkedIn post, X thread, newsletter blurb, short video and SEO article rarely get made.

Primary user: a solo founder or a 1 to 3 person marketing team that produces about one strong piece a week.

Job to be done: when I finish a long piece, I want channel-ready drafts in my voice that I can review in minutes, so the piece reaches every channel my audience uses without another afternoon of rewriting.

What they fear: a confident AI draft with a made-up statistic going out under their name. That fear shaped the product more than any feature request.

Evidence used: this is a problem framing from public marketer discussions and my own content workload, not a formal interview study. The engine was then run on real sources (see Results).

Goals and success metrics

North star: share of drafts a human publishes with light edits (under 25% of words changed). If people rewrite most of a draft, the engine saved typing, not thinking.

Metric	Type	Target for v1
Run time, submit to 5 drafts on the board	Guardrail: latency	Under 2 minutes
Drafts passing the quality gate first time	Guardrail: accuracy	4 of 5 or better
Numbers in drafts that are not in the source	Guardrail: accuracy	0 reach a human unflagged
LLM cost per source	Guardrail: cost	Under $0.25 at list prices
Words changed before publishing	North star	Under 25%

Solution and scope

One n8n workflow: a form takes one source, five channel drafts land on an approval board, and a person decides what ships.

In v1

- Three inputs: blog URL (scraped to markdown), audio file URL (transcribed), or pasted transcript
- A fact-only content brief extracted first
- Five drafts: LinkedIn post, X thread, newsletter section, 45 to 60 second video script, SEO blog draft
- Brand voice as a form field, with a plain default voice
- A code-based quality gate on every draft
- An approval board (n8n Data Table) with status, gate notes and token counts

Deliberately left out

- Direct YouTube download: fragile, a terms-of-service grey area, and nearly every video already has a transcript to paste
- Auto-posting to Buffer or LinkedIn: one confidently wrong post costs more than a two-minute review
- Image and thumbnail generation: separate problem, separate quality bar
- A public form URL: anyone with the link could spend the AI budget, so the form stays unpublished until it has authentication

Design

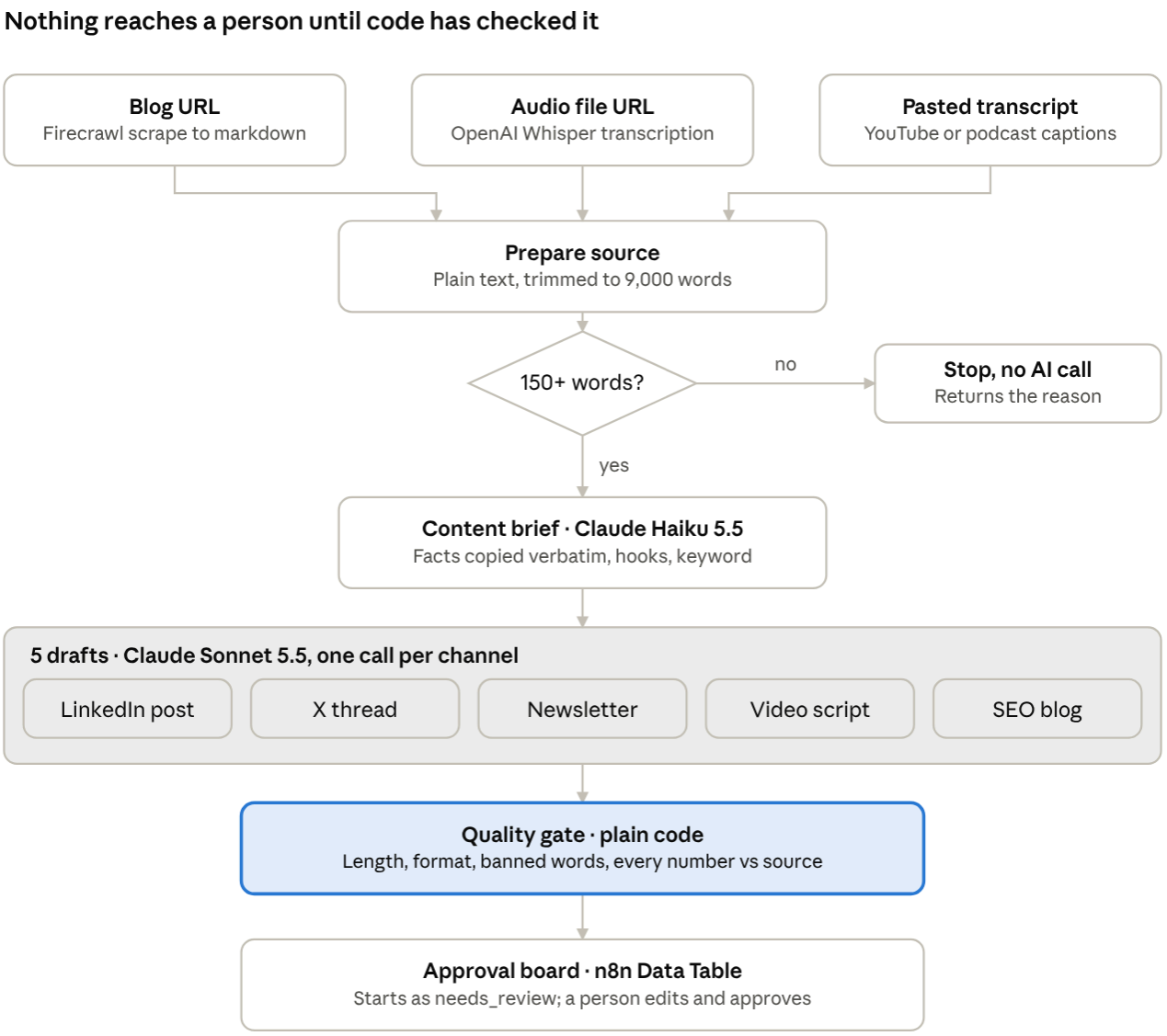
The whole user flow is three screens: a form, a wait of about a minute, and a review board.

1. **Intake form:** source title, source type (transcript, blog URL or audio URL), the URL or pasted text, and optional voice notes. Confirmation says when drafts will arrive.
2. **Approval board:** one row per draft with channel, word count, gate result, gate notes, token counts and a status that starts as `needs_review`. The reviewer edits into `final_text` and sets the status to approved or rejected.
3. **Publishing:** done by a person, by hand, in v1.

Failure and empty states

Situation	What the engine does	Why
Source under 150 words	Stops before any AI call and returns a plain reason	Five pieces from a tiny source forces the model to invent
Brief comes back as broken JSON	Passes the raw brief text through instead of failing	One bad parse should not kill five drafts
A draft breaks a rule	Still lands on the board, marked failed, with the exact reason	The reviewer decides; hiding drafts hides information
A number in a draft is not in the source	Flagged by value in gate notes	Invented statistics are the most damaging error
Model unsure of a detail	Writes `[CHECK: …]`, which the gate flags	A visible gap beats a confident guess

Architecture



AI Content Engine · n8n workflow, 19 nodes

Three inputs converge on one plain-text source. A word-count guard stops tiny sources before any AI spend. One cheap model writes the brief, a stronger model writes each channel, and plain code checks every draft before it reaches the board.

Component	Tool	Why this one
Orchestration	n8n (cloud)	Visual, versioned, exportable as JSON
Blog scraping	Firecrawl	Returns clean markdown with nav and ads removed
Transcription	OpenAI Whisper	Handles podcast and video audio
Brief	Claude Haiku 5.5	Extraction is structured and cheap

Component	Tool	Why this one
Drafts	Claude Sonnet 5.5	Tone and format need the stronger model
Quality gate	JavaScript Code node	Deterministic, free, explainable
Approval board	n8n Data Table	Lives in the same instance; no extra vendor

AI decisions

Brief first, drafts second. One extraction call builds a JSON brief (core message, key points with evidence, facts copied verbatim, hook angles, lesson, SEO keyword). Every draft call gets the brief and the full source, so drafts share one reading of the source instead of five.

One call per channel. Each format gets its own spec and the model's full attention, and one failed draft never takes the others down. It costs more input tokens than one big call; the cost table in Results shows what that trade-off costs.

Rules in code beat rules in prompts. The prompt asks for limits and no invented numbers. The model mostly listens. The quality gate then checks, in plain JavaScript:

- length and format per channel (hook length, hashtags, 280-character posts, subject line, H2 count, meta description length, video script sections)
- banned hype words and em dashes
- every number in a draft against the source text; any number not found is listed by value
- `[CHECK]` placeholders left by the model

Human in the loop by default. Nothing posts automatically. The board is the product's main screen.

Model choice. The n8n AI gateway rejected Sonnet 4.5 and Haiku 4.5 on this account, so I probed eight model IDs with a one-word prompt. Four worked. The brief uses Claude Haiku 5.5 (extraction is a cheap, structured task); drafts use Claude Sonnet 5.5 (tone and format need the stronger model).

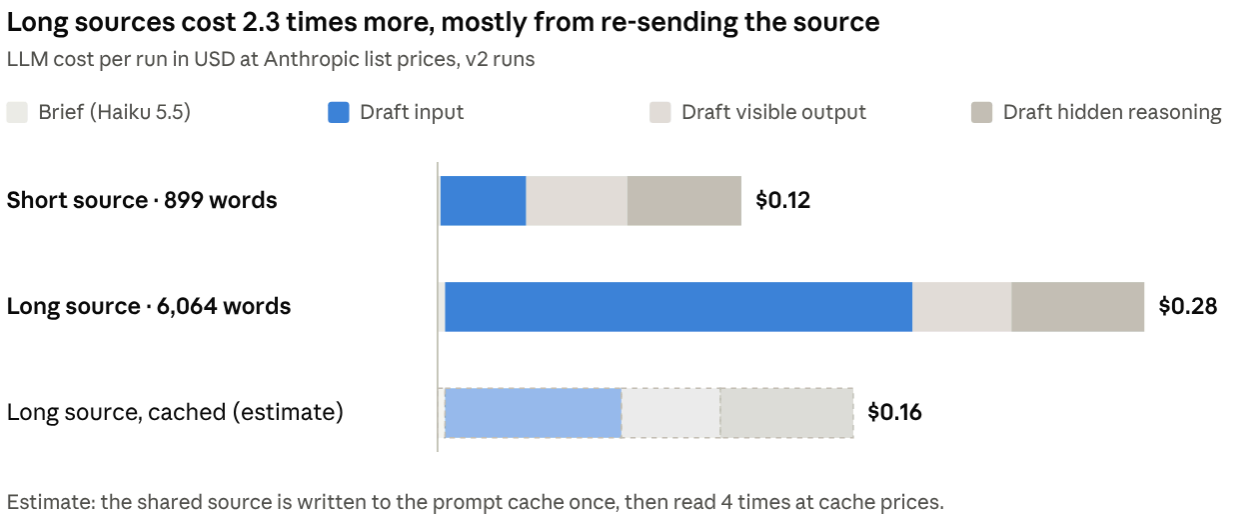
Privacy and security. Sources are processed only by the model and scraping providers in the workflow. Data stays in my own n8n instance. The form is not published, because a public form would let anyone spend the AI budget; it needs authentication before it goes live.

Results

All figures are **real**: five runs on November 1, 2026, logged by the workflow itself. Costs are logged tokens times Anthropic list prices; the cached figure is labelled as an **estimate**.

Run	Version	Source	Time	Gate pass	LLM cost	What the gate flagged
1	v1	My 899-word article, pasted	72 s	3 of 5	$0.12	Video CTA asks for a handle; SEO keyword missing from truncated brief
2	v1	6,064-word GitLab handbook page, by URL	88 s	4 of 5	not split by model in v1 logs	SEO draft `[CHECK]` placeholder
3	v1	28-second public-domain audio clip	2.5 s	no drafts	$0	Transcribed to 67 words; the guard stopped the run
4	v2	Same 899-word article	67.5 s	4 of 5	$0.12	SEO draft asked for real results instead of inventing them
5	v2	Same 6,064-word page	81 s	4 of 5	$0.28	X post 4 ran 281 characters; the prompt said under 270

Across 20 drafts: 15 passed first time (75%; 8 of 10 on v2). The number check found **0** invented numbers. Every flag pointed at something a reviewer should fix.



n8n execution logs, runs 4 and 5 · tokens × Anthropic list prices, Nov 1 2026

On short sources most of the cost is output, and over half of that is hidden reasoning. On long sources the five draft calls each re-send the whole source, so input becomes two thirds of the bill. Caching the shared source is the next fix.

Dogfooding: the engine wrote this project's post

I fed the engine a 630-word results write-up and published its LinkedIn draft.

Run	Gate version	Time	Gate pass	LLM cost	What happened
6	v2	71 s	0 of 5	$0.12	All 5 drafts flagged because they quoted the phrase "[CHECK] note" from the source: a gate bug, not a draft problem
7	v3	75 s	4 of 5	$0.13	Gate now matches only `[CHECK:` placeholders. The one flag was another edge case: the video script shows "[CHECK: real timing and pass-rate numbers]" on screen as a quote. A gate cannot tell intent from text alone, which is why a human still reviews

North star, first reading: I changed 48 of 236 words (20%) in the run 7 LinkedIn draft before publishing: adding the case-study line, a cost qualifier and a clearer number claim. One draft is a sample of one; the metric needs weeks of posts to mean anything.

Learnings and next steps

What I learned

- **Hidden reasoning is a budget line.** Over half of draft output tokens were reasoning, and in v1 it silently ate the brief's token budget. Log it from day one.
- **Rules in code catch what prompts miss.** The prompt said under 270 characters; one post came back at 281. Only code noticed.
- **Test the tester.** My own gate produced five false alarms the first time it read its own write-up. A unit test now covers that case.
- **The best failure state is an honest gap.** In 4 test drafts the model wrote a `[CHECK: …]` note instead of inventing a number. That behaviour is designed in, not luck.

Next steps

- [ ] Cache the shared source across the five draft calls (estimated $0.28 to $0.16 on long sources)
- [ ] Put basic authentication on the form, then publish it
- [ ] Track edit share for every approved draft, per channel, for four weeks
- [ ] Measure engagement on posts the engine drafted versus posts written by hand
- [ ] Add a scheduled step that sends approved drafts to Buffer

Assets and sources

- n8n workflow: "AI Content Engine" (19 nodes, unpublished) and the "Content Engine Approval Board" data table, in my n8n instance
- Repo package: workflow export, quality gate code, 6 unit tests, prompts, run data
- LinkedIn carousel (10 slides, PDF) and a 60-second demo video
- Test sources: my own 899-word article; GitLab Handbook: Product Management (CC BY-SA 4.0, test input only, not republished); JFK Rice University speech excerpt (public domain)
- Prices: Claude API pricing, checked November 1, 2026